



Compiladores

Aula 11: Análise sintática ascendente

Prof. Alex Torquato Souza Carneiro
alex.carneiro@impacta.edu.br

Agenda

- Parsing bottom-up
- Handles
- Modelo shift-reduce
- Prefixos viáveis
- Analisadores LR

Parsing bottom-up

- Na análise LL(1), expandimos o símbolo inicial e antecipamos a derivação.
- Com a estratégia de parsing bottom-up, começamos pelos tokens e reconstruímos a estrutura até a raiz.
- A ideia da análise ascendente é reconhecer a entrada e, progressivamente, agrupar fragmentos em não-terminais.



Parsing bottom-up

- Redução e derivação mais à direita reversa

Produções:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Entrada:

$id * id$

Sequência possível de reduções:

$$\begin{aligned} id * id &\Rightarrow F * id \Rightarrow T * id \\ &\Rightarrow T * F \Rightarrow T \Rightarrow E \end{aligned}$$

Atenção

Reduzir não é substituir qualquer subcadeia possível.

É preciso encontrar o fragmento que corresponde ao próximo passo da derivação reversa: o handle.

Handles

- Handles dão suporte à identificação do fragmento certo para reduzir.
- Handle é uma subcadeia que coincide com o corpo de uma produção e cuja redução desfaz o próximo passo da derivação mais à direita.

Forma sentencial	Handle	Redução	Qual produção, se desfeita, recupera a forma sentencial anterior?
$id_1 * id_2$	id_1	$F \rightarrow id$	
$F * id_2$	F	$T \rightarrow F$	
$T * id_2$	id_2	$F \rightarrow id$	
$T * F$	$T * F$	$E \rightarrow T * F$	

Modelo shift-reduce

- O modelo shift-reduce combina a pilha e o buffer com o objetivo de reconhecer o contexto que auxilia a escolha da regra de produção.

SHIFT: empilha o próximo token e avança a entrada

REDUCE: remove β e empilha A em $A \rightarrow \beta$

ACCEPT: entrada reconhecida por completo

ERROR: não há ação válida para a configuração

Modelo shift-reduce

- Uma implementação real mantém estados além dos símbolos.
- Durante cada redução, o parser também pode construir ou combinar nós da AST.

Produções:

$E \rightarrow E + T \mid T$

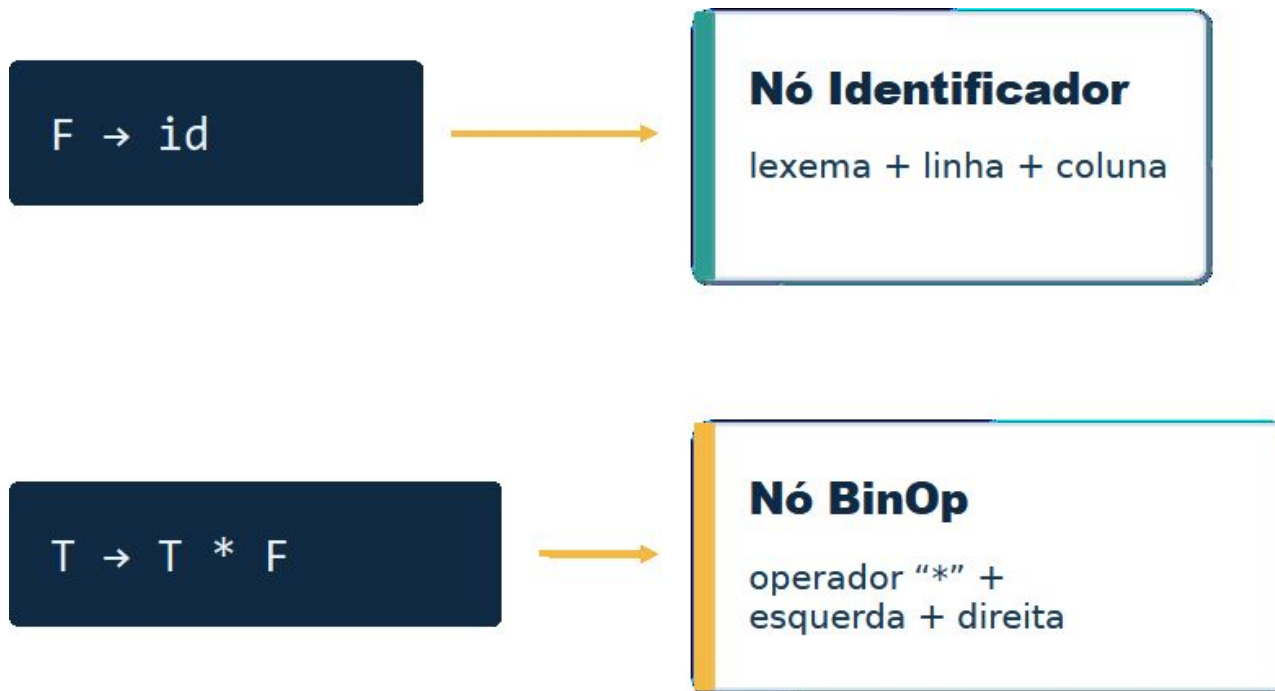
$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

Pilha	Entrada	Ação
\$	id * id \$	shift id
\$ id	* id \$	reduce $F \rightarrow id$
\$ F	* id \$	reduce $T \rightarrow F$
\$ T	* id \$	shift *
\$ T * id	\$	reduce $F \rightarrow id$
\$ T * F	\$	reduce $T \rightarrow T * F$
\$ E	\$	accept

Modelo shift-reduce

- As reduções são pontos naturais para criar a representação estrutural do programa.
- AST preserva a estrutura relevante para a análise semântica e a geração de código.



Prefixos viáveis

- Prefixo de uma forma sentencial à direita que não ultrapassa o handle.
- Ele representa exatamente o contexto que a pilha pode conter.
- Item LR(0): Uma produção com um ponto indicando quanto do lado direito já foi reconhecido.

$$[A \rightarrow \alpha \cdot \beta]$$

- Antes de um símbolo: pode haver shift
- Antes de um não-terminal: aplicar CLOSURE
- No final: redução disponível

Prefixos viáveis

- CLOSURE e GOTO: a base do autômato
- Gramática estendida:

$$E' \rightarrow E$$

$$E \rightarrow E + T$$

$$E \rightarrow T$$

- Conjunto inicial I_0 :

$$[E' \rightarrow \cdot E]$$

$$[E \rightarrow \cdot E + T]$$

$$[E \rightarrow \cdot T]$$

CLOSURE(I)

Inclui itens de produções cujo ponto antecede um não-terminal.

GOTO(I, X)

Move o ponto sobre X e calcula o fechamento do conjunto resultante.

Analísadores LR

- Uma forma de aumentar as evidências de contexto consiste no uso de analisadores LR.

Método	Contexto usado	Leitura prática
LR(0)	itens sem lookahead	simples; muitos conflitos
SLR(1)	FOLLOW para reduções	autômato LR(0) + contexto global
LR(1)	lookahead nos itens	mais poderoso; tabelas maiores
LALR(1)	combina estados compatíveis	bom equilíbrio entre poder e tamanho

Ideia central: a pilha de estados e as tabelas orientam as ações sem retrocesso.

Analísadores LR

- Conflitos: quando a tabela não decide.
- Shift / reduce

A mesma configuração permite:

- transferir o próximo token (shift); ou
- reduzir o conteúdo da pilha (reduce).

Exemplo clássico: associar `else` ao `if` mais próximo.

- Reduce / reduce

Duas reduções diferentes parecem possíveis.

Geralmente indica gramática ambígua, especificação incompleta ou contexto insuficiente.

Aplicação ao parser MINIC



A AST não precisa repetir todos os símbolos da gramática: ela deve preservar a estrutura relevante.